

ForensiX AI

An AI-Powered Digital Memory Search Engine and Cybersecurity Investigation Platform

Detailed Project Explanation & Technical Reference

This document explains the supplied ForensiX AI design from problem statement through collection, centralized storage, correlation, natural-language investigation, behavioral analysis, integrity verification, reporting, dashboard design, limitations, research contribution and evaluation.

1. Project Definition

ForensiX AI is a Windows desktop digital-investigation platform for a single endpoint. It continuously collects supported security-relevant telemetry—Windows Event Logs, browser history, file activity, USB device events and Windows Defender alerts—then centralizes those events into a chronological database. AI correlation links related events into an incident timeline. A natural-language interface allows investigation using plain English. A personal behavioral baseline flags meaningful deviations, while SHA-256 hash chaining makes post-collection modification or deletion detectable. An investigator dashboard brings the results together.

The project's stated motivation is to make investigation of a single computer more accessible to individuals and small organizations that do not have a dedicated security team. The supplied project reference scopes the work to an eight-month academic timeline.

2. Problem Statement

- Security-relevant evidence is fragmented across Windows Event Logs, browser history, file activity, USB events and antivirus alerts.
- Different sources use different formats and normally require separate tools and investigation processes.
- Raw forensic/security data is technical and difficult for non-experts to interpret.
- Investigators need to connect events across sources to understand an incident as a sequence rather than isolated records.
- Collected logs need an integrity mechanism so post-collection changes can be detected.

3. Objectives

- **Collection:** capture the five stated telemetry categories from one endpoint.
- **Centralization:** normalize heterogeneous records into one chronological database.
- **Correlation:** link related events into a coherent incident timeline.
- **Querying:** provide plain-English investigation through a local LLM.
- **Reporting:** generate structured, human-readable incident summaries.
- **Integrity:** use hash-chained tamper-evident logging.
- **Verification:** identify the point at which the hash chain becomes invalid.
- **Baselining:** learn normal activity and flag meaningful deviations.
- **Visualization:** provide a consolidated investigator dashboard.

4. Complete Architecture

Windows Endpoint → Source Collectors → Event Normalization → Central Database → Hash Chain → Correlation + Behavioral Analysis → Timeline → Natural-Language Query → Findings → Report + Dashboard

The architecture is deliberately endpoint-focused. It is not a network-wide SIEM and is not intended to replace full forensic acquisition/recovery suites.

5. Data Collection Layer

Collector	What it provides	Reference implementation direction
Windows Event Logs	Security/system events and other supported Windows event records	win32evtlog
Windows Defender	Defender/security alerts that can be correlated with other events	win32evtlog / supported Windows interfaces
Browser history	Historical browser activity from supported browser databases	Direct SQLite read
File activity	Supported file create/modify/delete and related filesystem activity	Watchdog
USB events	Supported USB device connection/activity information	WMI

The supplied methodology recommends dedicated collectors because each source exposes data differently. The collectors then normalize records into a common schema and write them to the central store.

Engineering considerations: browser history databases may be locked while a browser is running; multiple collectors writing directly to one database can create write-lock contention; continuous monitoring adds CPU/disk overhead. A write queue/single-writer pattern is a practical implementation approach.

6. What the Project Can Actually Observe

- Supported Windows/system events and their available metadata.
- Browser artifacts that are exposed in the browser history database.
- Supported filesystem activity such as file creation, modification, deletion, rename/move and metadata.
- Supported USB connection/device events.
- Windows Defender/security alerts.
- Correlations between these records when the available evidence supports the relationship.

It should **not** be described as seeing everything a user does. For example, a file monitor does not automatically reveal every action performed inside an application, and a normal browser-history collector cannot guarantee recovery of deleted browser history.

7. Event Normalization

Normalization converts different source formats into a common event representation so one database and one investigation interface can work across all sources.

Field	Purpose
timestamp	When the event occurred
source	Origin such as Windows, browser, file monitor, USB or Defender
event_type	Type of activity/record
user	Associated user when available
process/application	Associated process/application when available
path / URL / device	Source-specific object
details	Additional normalized attributes
raw reference	Reference to the source artifact where applicable
hash / previous_hash	Integrity-chain information
severity / deviation	Analysis output where applicable

8. Centralized Database

The database is the common investigation surface. Instead of searching five unrelated sources, the investigator can query one chronological collection. Centralization also makes cross-source correlation possible.

A useful design principle is to retain enough source information to trace a normalized event back to its original artifact. The AI's interpretation should not replace the underlying evidence.

9. AI Correlation Engine

Correlation is the step between *having logs* and *understanding an incident*. It attempts to identify relationships among events using time, source, process/application, user, object and other available attributes.

Example sequence: USB device connected → unusual file activity → file modification/deletion → Defender alert → related process activity. If the evidence supports the relationship, the system can group these events into a coherent incident sequence.

The project should preserve references from correlated/high-level findings back to the events that support them. This provides traceability and reduces the risk of an AI-generated story becoming disconnected from evidence.

10. Forensic Timeline

The timeline is the central investigation view. It orders normalized events chronologically and shows how separate records fit into an incident. The research direction behind this feature is timeline-based event reconstruction: moving from low-level events toward meaningful high-level events while preserving provenance.

Raw events → normalized events → temporal ordering → correlation → incident sequence

11. Natural-Language Investigation / Local LLM

The project reference proposes a locally running LLM through Ollama. The user asks a plain-English question; the system interprets the intent, retrieves relevant records from the centralized database, and summarizes the evidence.

User question → LLM intent interpretation → structured database retrieval → evidence → LLM explanation

Why local? Local inference keeps sensitive logs on-device rather than automatically sending them to a cloud model. The trade-offs are RAM/CPU usage, latency and potentially lower accuracy for small models.

Forensic safety principle: the LLM is an investigation assistant, not the evidence itself. Answers should expose supporting events and distinguish observed facts from interpretation.

12. Personal Behavioral Baseline

The baseline module learns normal behavior for the particular endpoint/user. The reference design describes a cold-start period and then a rolling baseline covering patterns such as active hours, common applications and regular domains.

The detector should use a confidence/severity/deviation score rather than a simple normal/abnormal decision. Example: activity at an unusual hour involving an uncommon application can receive a higher deviation score if it significantly differs from the established baseline.

Limitations: a cold start is required; a short or unrepresentative training window can cause false positives or false negatives; and the baseline adds model-maintenance complexity.

13. SHA-256 Hash-Chained Tamper-Evident Logging

ForensiX AI does not need a full blockchain network for its integrity objective. Each event's SHA-256 hash incorporates the event content and the previous event's hash.

$$H0 \rightarrow H1 = \text{SHA256}(\text{Event1} + H0) \rightarrow H2 = \text{SHA256}(\text{Event2} + H1) \rightarrow H3 = \text{SHA256}(\text{Event3} + H2)$$

If Event2 is modified, its recalculated hash differs and the chain relationship fails. If a record is removed, the expected previous-hash relationship also fails. The result is **tamper evidence**, not a guarantee that the database cannot be destroyed.

The supplied reference explicitly warns that this should be called hash chaining rather than blockchain. It also notes that if the entire local database is deleted or compromised, the chain cannot recover the lost data; it can only detect integrity problems.

14. Integrity Verification

Verification recomputes the chain and compares every stored hash and previous-hash relationship. The application can report the first invalid record/index and mark the evidence set as having an integrity failure from that point.

15. Automated Incident Report

The report module turns investigation results into a structured human-readable summary. A strong report should contain the investigation question, time range, important events, correlated sequence, anomaly indicators, supporting evidence references, hash-chain verification status and known limitations.

AI-generated prose should remain separate from observed evidence. The report should make it possible for a reviewer to inspect why a finding was produced.

16. Investigator Dashboard

- **Overview:** system/collector status, recent alerts and investigation summary.
- **Timeline:** chronological events with filtering by source, time, severity and event type.
- **Event details:** normalized information and supporting source/reference data.
- **Investigation:** natural-language question interface.
- **Anomalies:** behavioral deviations and severity/confidence.
- **Integrity:** hash-chain status and location of a detected break.
- **Reports:** generated incident summaries.

17. Real-Time Incident Flow

1) Event occurs → 2) appropriate collector captures it → 3) event is normalized → 4) event enters central database → 5) hash-chain information is recorded → 6) correlation/baseline analysis runs as applicable → 7) related events appear in timeline → 8) investigator asks questions → 9) findings are presented with supporting evidence → 10) report is generated and integrity status is included.

18. If ForensiX AI Was Not Installed Before the Incident

ForensiX AI cannot recreate telemetry that was never recorded. A future/offline forensic mode can, however, import historical artifacts that still exist on the system or in an acquired evidence set, such as supported Windows Event Logs, browser artifacts, Defender history and other supported application/filesystem artifacts.

Suspect computer/evidence set → historical artifacts → ForensiX import → normalization → hash at ingestion → database → timeline/correlation → investigation

Deep deleted-file recovery/data carving, bit-level disk imaging and RAM forensics are explicitly outside the current project scope.

19. Deleted Data and Full Forensics — Current Boundary

Real forensic teams may recover some deleted information by examining filesystem metadata, unallocated space, journals, Recycle Bin artifacts, backups/shadow copies and other remnants. Recovery becomes uncertain when information has been overwritten, and SSD TRIM can further complicate recovery. ForensiX AI deliberately does not implement this specialized recovery pipeline in the current scope.

This boundary is important: **ForensiX AI is primarily an evidence collection, correlation, investigation and integrity platform—not a complete disk-recovery suite.**

20. Security and Privacy Design

- Use only supported and authorized telemetry collection.
- Use local LLM inference where practical so sensitive evidence remains on-device.
- Protect the local database and restrict administrative operations.
- Preserve source/raw references where possible.
- Hash-chain collected records and provide on-demand verification.
- Record collection/source metadata where useful for reproducibility.
- Do not treat an AI narrative as proof without supporting evidence.

21. What Is In Scope / Out of Scope

In scope	Explicitly out of scope
Single Windows endpoint	Disk imaging / bit-level acquisition
Windows Event Logs	Deleted-file recovery / data carving
Browser history	RAM forensics
File activity	Network-wide / multi-device monitoring
USB events	Legal admissibility guarantees
Windows Defender alerts	
Centralized database	
AI correlation + timeline	
Natural-language local LLM	
Behavioral baseline	
Hash-chain integrity + verification	
Reports + investigator dashboard	

22. Research Contribution

The strongest academic framing is not “no existing tool does this.” The contribution is the **integration and experimental evaluation** of multiple forensic/security capabilities in a focused single-endpoint platform: heterogeneous endpoint telemetry collection, normalization, cross-source event correlation, timeline reconstruction, local-LLM investigation, personalized behavioral anomaly detection and lightweight tamper-evident integrity verification.

A suitable research question is: *Does AI-assisted correlation and natural-language investigation of heterogeneous endpoint telemetry improve the efficiency and interpretability of single-endpoint digital forensic investigation while preserving evidence traceability and integrity?*

23. Evaluation Plan

- **Collection coverage:** percentage/types of controlled test events captured correctly.
- **Timeline accuracy:** whether known incident sequences are reconstructed in the correct order.

- **Correlation quality:** precision/recall or expert/user-rated correctness of event grouping.
- **Natural-language accuracy:** whether questions retrieve the correct evidence and avoid unsupported claims.
- **Anomaly detection:** false positives, false negatives, precision, recall and detection delay.
- **Integrity:** ability to detect modified/deleted records and identify the first broken chain link.
- **Performance:** CPU, RAM, disk overhead and LLM response latency.
- **Usability:** investigation task completion time and user feedback.

24. Feature → Objective → Problem Traceability

Problem	Objective	Feature
Fragmented evidence	Collection + Centralization	Five-source collectors + unified chronological database
Disconnected events	Correlation	AI correlation + incident timeline
Technical investigation	Querying	Natural-language local-LLM interface
Difficult communication	Reporting	Automated human-readable report
Evidence trust	Integrity	SHA-256 hash-chained logging
Need to detect manipulation	Verification	On-demand chain verification
Unknown normal behavior	Baselining	Personal adaptive behavioral baseline
Investigator complexity	Visualization	Consolidated dashboard

25. Research Literature Areas

Area	Representative paper(s) from the project reference	Role
Timeline reconstruction	An Automated Timeline Reconstruction Approach for Digital Forensic Investigations	Timeline + event reconstruction
LLM forensics	ForensicLLM: A Local Large Language Model for Digital Forensics	Local LLM design
LLM timeline analysis	Towards a standardized methodology for evaluating LLM-based digital forensic timeline analysis	Natural-language forensic analysis
LLM security automation	AI-Augmented SOC: A Survey of LLMs and Agents for Security Automation	Summarization/triage/reporting context
Endpoint forensics	Threat identification-based collection of forensic data from endpoint devices	Endpoint collection architecture
Browser forensics	Advancing Web Browser Forensics: Critical Evaluation of Tools	Browser artifact collection
Behavioral anomaly detection	Anomaly detection in Windows event logs for ransomware identification	Windows-log anomaly detection
Evidence integrity	B-CoC: A Blockchain-based Chain of Custody for Evidence Management	Cryptographic evidence integrity concept
Forensics + blockchain	SoK: Blockchain Solutions for Forensics	Chain-of-custody research context

26. Final Mental Model

ForensiX AI can be remembered as:

COLLECT → CENTRALIZE → NORMALIZE → PRESERVE → CORRELATE → DETECT → ASK → EXPLAIN → VERIFY → REPORT

- **Collect:** gather supported endpoint evidence.
- **Centralize:** store it in one place.
- **Normalize:** make heterogeneous records comparable.
- **Preserve:** hash-chain records for tamper evidence.
- **Correlate:** connect related activity.
- **Detect:** identify deviations from normal behavior.
- **Ask:** investigate in plain English.
- **Explain:** connect findings to supporting events.
- **Verify:** test integrity.
- **Report:** generate a structured investigation summary.

27. Source Basis

This document is grounded primarily in the supplied **ForensixAI.pdf / Consolidated Project Reference Document**, which defines the project summary, problem, objectives, scope, methodologies, pros/cons, research references and feature-to-objective traceability. Additional explanatory wording in this document is clearly framed as implementation/design guidance where it goes beyond the supplied reference.